

Q.1 (50%): Memory Management

A. Consider a system of **16 GB** (2^{34} B) of memory with a frame size of **8 KB** (2^{13} B) and a process **P** with a logical address space of size **450 KB**. Assume that page **N** of this process is mapped to frame **N+2000**. Answer the following questions showing your work.

1. What is the physical address of the logical address **345678**?

$$1 - VL = (345678 \div 8192, 345678 \bmod 8192) = (42, 1614)$$

Page number offset

$$2 - VP = (2042, 1614)$$

$$3 - \text{Physical address} = 8192 \times 2042 + 1614 = 16737870$$

2. What is the amount of internal fragmentation, if any?

$$8192 - ((450 \times 2^{10}) \bmod 8192) = 8192 - 2048 = 6144 \text{ B}$$

3. How many pages are needed to represent this process?

$$\frac{450 \text{ KB}}{8 \text{ KB}} = \lceil 56.25 \rceil = 57 \text{ pages}$$

4. How many bits are needed to represent the offset?

$$8 \times 2^{10} = 2^3 \times 2^{10} = 2^{13} \rightarrow 13 \text{ bits.}$$

B. Consider the following segmentation table for the process **P** with zero internal fragmentation. Answer the following questions showing your work.

Segment Number	Base Address	Limit
0	12100000	5400
1	14200000	7600
2	16300000	16500 ←
3	18400000	2800

1. What is the physical address of the logical address **16500**?

$$16500 > 5400 \rightarrow 16500 - 5400 = 11100 > 7600$$

$$11100 - 7600 = 3500 < 16500 \rightarrow \text{Segment } (2) \rightarrow (2, 3500)$$

$$\text{Physical address} = 16300000 + 3500 = 16303500$$

2. What is the virtual logical address of the physical address **14204567**?

$$14204567 - 14200000 = 4567$$

3. What is the total size of the process address space?

$$5400 + 7600 + 16500 + 2800 = 32300$$

C. Consider a memory of size 32KB. This memory is allocated in increments of 2KB and the memory is currently represented as follows:

Free (6KB)	Occupied (8KB)	Free (10KB)	Occupied (4KB)	Free (4KB)
---------------	-------------------	----------------	-------------------	---------------

Use the various contiguous allocation schemes (First-fit, Best-fit, and Worst-fit) to show the memory representation after satisfying the allocation requests for processes requesting sizes as given below.

Process	A	B	C	D
Requested size (in KB)	3.5	2.3	5.2	1.6

First-fit	A (4KB)	D (2KB)	Occupied (8KB)	B (4KB)	C (6KB)	Occupied (4KB)	Free (4KB)
Best-fit	B (4KB)	D (2KB)	Occupied (8KB)	C (6KB)	Free (4KB)	Occupied (4KB)	A (4KB)
Worst-fit	B (4KB)	Free (2KB)	Occupied (8KB)	Free (4KB) (A)	C (6KB)	Occupied (4KB)	D (2KB) Free (2KB)

The amount of internal fragmentation (in KB) for Best-fit algorithm is $0.7 + 0.4 + 0.8 + 0.5 = 2.4 \text{ KB}$

The amount of external fragmentation (in KB) for Worst-fit algorithm is $2 \text{ KB} + 2 \text{ KB} = 4 \text{ KB}$

D. Consider a system in which a process is allocated three frames in memory. Given the page requests of this process as "7, 5, 3, 1, 5, 2, 4, 6, 4, 2, 5, 7, 1, 5, 3", trace the utilization of the three frames by this process and find the number of page faults when using the Least Recently Used (LRU) replacement algorithm.

LRU Replacement															
Frame	7	5	3	1	5	2	4	6	4	2	5	7	1	5	3
0	7	7	7	1		1	4	4			4	7	7		3
1		5	5	5		5	5	6			5	5	5		5
2			3	3		2	2	2			2	2	1		1
Number of page faults = 11															

Q.2 (30%): Multiple Choice

1. What is the output of the following program?

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main() {
    pid_t child;
    int status;
    child = fork();
    switch (child) {
        case -1:
            perror("fork");
            exit(1);
        case 0:
            exit(2);
            break;
        default:
            while (waitpid(child, &status, WNOHANG) == 0)
                sleep(1);
            printf("%d\n", WEXITSTATUS(status));
    }
    return 0;
}
```

Handwritten notes:
child →
Parent →

- a) 0
- b) 1
- c) 2
- d) None of the above

2. What is the output of the following program?

```
#include<stdio.h>
#include<unistd.h>
int main() {
    int x = 10, y = 80;
    y = y + x;
    pid_t c = fork();
    if (c == 0) {
        x += y;
        printf("%d%d", x, x - y);
    }
    else {
        wait(NULL);
        printf("%d%d", x, y);
    }
    return 0;
}
```

Handwritten calculations:
y = 80 + 10 = 90
Child: x = 10 + 90 = 100
Parent: x = 10, y = 90
Child output: 100 100
Parent output: 100 90

- a) 100109090
- b) 100101090
- c) 109010010
- d) 1001010090

3. Assume that the PID of the program below is 53720 and PIDs of consequent processes are assigned sequentially by the kernel in the order 53721, 53722 ... etc. Which of the following is the correct output of the program below?

```
#include<stdio.h>
#include<unistd.h>
#include<signal.h>
```

```
void h(int s){
    printf("[%d]", getpid());
}
```

```
void main(){
    signal(SIGKILL, h);
    pid_t c;
    c=fork();
```

```
    if(c == 0){
        sleep(1);
        printf("[%d]", getpid());
    }
```

```
    if(c != 0) {
        kill(c, SIGKILL);
        wait(NULL);
    }
```

```
    printf("[%d]", getpid());
```

Shared

- a) [53720]
- ☒ b) [53721][53720]
- c) [53721][53721][53720]
- d) [53721][53721][53721][53720]

4. How many times "CMPS405" will be printed in this program?

```
#include<stdio.h>
#include<stdlib.h>
#include<unistd.h>
int main() {
    if (fork()==0) {
        if( execl("/bin/ls", "ls", NULL) == -1){
            perror("execl");
            exit(1);
        }
        printf("CMPS405\n");
    }
    return 0;
}
```

a) 0

☒ b) 1

c) 2

d) None of the above

5. In the Additional-Reference-Bits Algorithm used by LRU, which of the following shift register values represents a page that would be the best choice for replacement?

- ☒ a) 00110011
- b) 01010101
- c) 10101010
- d) 11001100

6. Which of the following is an advantage of the Layered Structure of OS Design?

- ☒ a) Defining appropriate layers data and operations is an easy task
- b) Direct user access to low-level facilities is allowed
- c) Very efficient in communicating requests between the layers
- d) Simplification of debugging and system verification

7. Which type of Operating System architecture allows all processors to communicate with each other by a shared memory?

- ☐ a) Symmetric multiprocessing
- b) Asymmetric multiprocessing
- c) Real Time
- ☒ d) Distributed

8. Which of the following is not true in the context of signals in Linux OS?

- ☒ a) kill is both a system command and as a system call
- b) All standard signals can be ignored and handled
- c) The command kill -l can be used to list all signals in the system
- d) Ctrl+c keyboard keys result in sending SIGTERM to the current active process in the shell

9. Which of the following describes a Zombie process?

- ☒ a) A process that its parent exited while it is still running
- b) A process that makes exit when its parent is not executing wait
- c) A process that is adopted by the system init process
- d) A process that replaces its memory space with a new program

10. Which of the following is not true about the pipe operation?

- a) The file descriptor 0 is used for reading while the 1 is used for writing
- ☒ b) If a write is made on a pipe that is full the data will be dropped
- c) If a pipe is empty the reader process suspended until more output is available
- d) It requires two pipes to implement the command line `ls | sort | wc -c`

Q.3 (20%): System Calls and IPCs (Note: you do not need to include any header file)

- A. Write a C program in which the child process writes to a pipe all processes running in the system, while the parent process reads from the pipe to count the number of processes. Your code should avoid using the function system.

```

void main() {
    int p[2];
    pipe(p);
    pid_t pid;
    pid = fork();
    if (pid > 0) {
        dup2(p[0], 0);
        close(p[1]);
        execlp("wc", "wc", "-l", NULL);
    }
    else {
        dup2(p[1], 1);
        close(p[0]);
        execlp("ps", "ps", NULL);
    }
}
    
```

10

- B. Write a C program that forks as many sibling children as needed to print a message entered by the user. Each process must strictly print one character of the message. In addition, you should make sure that the characters of the message are printed in the correct order regardless of the order of execution of these processes. Avoid having any premature or zombie processes.

```

void main() {
    pid_t pid; char msg[n]; int n;
    printf("Enter a message");
    gets(msg);
    for (int i=0; i < n; i++) {
        if (!fork()) {
            printf("%c", msg[i]);
            exit(0);
        }
        else {
            wait(NULL);
        }
    }
}
    
```

6

Good Luck